XM · CLASES 9 y 10

Soluciones de Analítica con Databricks

Clases 9 y 10: MLOps en acción: experimentar, medir y registrar

Docente: Manuela Larrea Gómez

Agenda de las clases 9 y 10

Sesión doble · 2 h 30 min · qué es MLOps, qué medir y para qué, qué ofrece Databricks, y verlo en acción

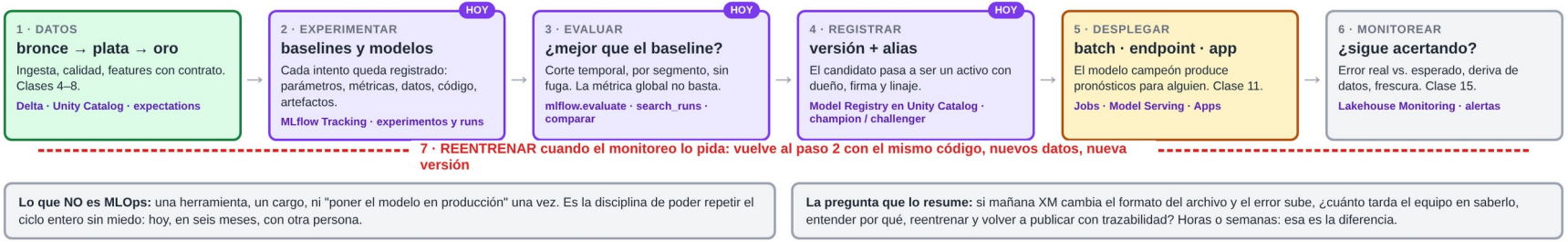
01	30 MIN	**Teoría, parte 1** ¿Qué es MLOps? El ciclo, la deuda técnica, niveles de madurez · ¿Qué medir y para qué? Cinco capas, métricas por tipo de modelo, segmentos
02	10 MIN	**Receso**
03	25 MIN	**Teoría, parte 2** ¿Qué ofrece Databricks? MLflow Tracking, evaluación honesta, Model Registry en Unity Catalog · resultados reales · sector
04	75 MIN	**En acción: notebook guiado + lab** Siete runs de cinco tipos de modelo · walk-forward · la fuga vista en el tracking · modelo registrado con @champion
05	10 MIN	**Cierre** Hito 5 abierto · tag s09 · así sería en Azure · próxima clase: serving y apps

¿Qué es MLOps? El ciclo de vida, no el momento de entrenar

Entrenar es un paso de siete. Lo que distingue a un modelo que opera de un notebook con buena métrica es todo lo demás

MLOps: el ciclo de vida de un modelo, no el momento en que se entrena

Entrenar es un paso de siete. Lo que distingue un modelo que opera de un notebook con buena métrica es todo lo demás, y se repite cada vez que los datos cambian.



Lo que NO es MLOps: una herramienta, un cargo, ni "poner el modelo en producción" una vez. Es la disciplina de poder repetir el ciclo entero sin miedo: hoy, en seis meses, con otra persona.

La pregunta que lo resume: si mañana XM cambia el formato del archivo y el error sube, ¿cuánto tarda el equipo en saberlo, entender por qué, reentrenar y volver a publicar con trazabilidad? Horas o semanas: esa es la diferencia.

Prueba ácida: si mañana XM cambia una columna del archivo, ¿cuánto tarda el equipo en notarlo, entender por qué, reentrenar y volver a publicar sabiendo qué modelo produjo cada número? Horas = MLOps. Semanas = no.

Clases 4–8

Paso 1: datos con contrato, calidad, features, tests.

Hoy (9 y 10)

Pasos 2, 3 y 4: experimentar, evaluar y registrar, con MLflow.

Clases 11–15

Pasos 5 a 7: desplegar, orquestar, monitorear, reentrenar.

Por qué hace falta: el código del modelo es lo más pequeño

Adaptado de Sculley et al., Hidden Technical Debt in Machine Learning Systems (NeurIPS 2015)

Recolección de datos

clase 4 · Auto Loader

Verificación

clase 5 · expectations

Extracción de features

clase 7 · oro

Configuración

clase 8 · conf/ + validar

Gestión de recursos

serverless · jobs

Código ML

← lo que se aprende en un curso de modelos

Herramientas de análisis

tracking · evaluación · hoy

Infraestructura de serving

clase 11

Gestión de procesos

jobs · bundles · clases 12–13

Monitoreo

clase 15

Verde: ya lo construyeron. Morado: hoy. Ámbar: lo que viene. Por eso los modelos llegan en la clase 9 y no en la 2.

Niveles de madurez: dónde está un equipo y a dónde llega este proyecto

Modelo de Google Cloud (MLOps: continuous delivery and automation pipelines in machine learning)

Nivel 0

Manual

Notebook → archivo → alguien lo pone en producción. Cada reentrenamiento es un proyecto. Nadie sabe con qué datos se entrenó el modelo de marzo.

Donde empiezan casi todos

Nivel 1

Pipeline de entrenamiento automatizado

Entrenar es una tarea programada con datos y modelos versionados. Reentrenar es un botón. Cada run deja rastro.

Donde queda este proyecto

Nivel 2

CI/CD del pipeline

El propio pipeline se prueba y se despliega con CI/CD por ambientes, como el código en la clase 8.

Piezas en clases 8 y 13

El paso más importante para salir del nivel 0 es el de hoy: que cada entrenamiento deje rastro.

Fuente: cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

¿Qué medir y para qué? Cinco capas, cinco destinatarios

Una métrica entra al proyecto cuando alguien puede decir qué decisión cambia si sube o si baja

Qué medir y para qué: cuatro capas, y cada una responde a una persona distinta

Una métrica sin un "para qué" es un número. Para cada una: quién la mira, qué decisión toma con ella y dónde queda registrada en Databricks.

Capa	Métricas (ejemplos de este proyecto)	Para qué: la decisión que habilita	Quién la mira	Dónde queda
Modelo · calidad	WAPE, MAE en GWh, error por segmento (regulado / no regulado, día, serie), AUC y recall en el clasificador, silhouette en clústeres	¿Promuevo este candidato? ¿Es mejor que el baseline donde importa (festivos, mercado regulado)?	Científico de datos, líder del equipo	MLflow: métricas del run; comparación entre runs
Modelo · honestidad	Corte temporal, walk-forward, diferencia entrenamiento vs. validación, prueba anti-fuga	¿La métrica es creíble o está inflada? ¿Se sostiene en el tiempo?	Científico de datos, revisor del PR	MLflow: runs anidados por fold; tags (fuga=true)
Datos	Filas, nulos, rango de fechas, versión Delta usada, distribución de features vs. entrenamiento (deriva)	¿El modelo está viendo lo que vio al entrenar? ¿Hay que reentrenar?	Ingeniero de datos, operación	log_input (tabla + versión) · expectations · Lakehouse Monitoring
Operación	Latencia y costo de inferencia, frescura del pronóstico, tasa de fallos del job, tiempo de reentrenamiento	¿Llega el pronóstico a las 6 a. m.? ¿Cuánto cuesta cada corrida?	Operación, plataforma	Jobs (duración, estado) · Serving (p95, errores) · alertas
Negocio	Error en los días que deciden el despacho, costo de sobre/sub-pronóstico, pronósticos publicados y corregidos	¿El modelo vale lo que cuesta? ¿Se puede defender ante el regulador?	Negocio, gerencia, auditoría	Tablas de oro (pronóstico vs. observado) · tableros · registro con linaje

Regla: una métrica entra al proyecto cuando alguien puede decir qué decisión cambia si sube o si baja. Si nadie decide nada con ella, es ruido en el tablero.

WAPE
error ponderado por energía: la métrica principal del pronóstico

por segmento
regulado / no regulado, día, serie: la global esconde

vs. baseline
una métrica sola no dice nada; comparada, sí

versión Delta
qué datos vio el modelo: medir también es anotar

Cada tipo de modelo tiene sus métricas, y su trampa

Hoy vemos los tres primeros en acción sobre los mismos datos; el cuarto, como referencia

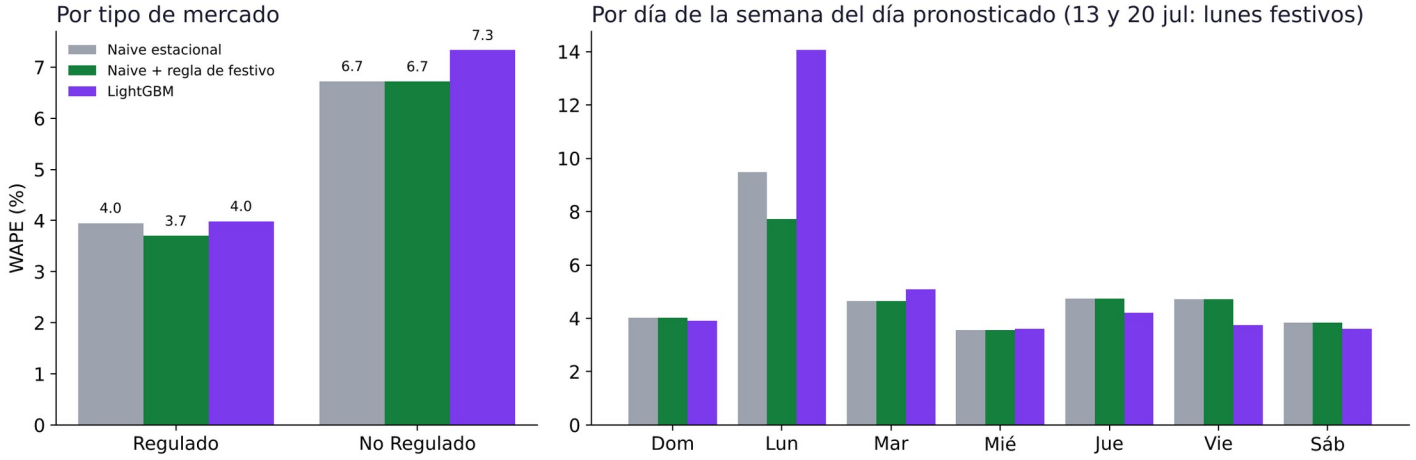
Tipo	Ejemplo de hoy	Métricas	La trampa	Artefacto que se guarda
Regresión (pronóstico)	Demanda de cada serie en t+7	WAPE, MAE en GWh, error por segmento	MAPE: explota con ceros y pesa igual 30 kWh que 35 GWh	Errores por serie, importancia
Clasificación	¿Caerá más de 10 % en 7 días?	AUC, precisión, recall	Exactitud: con 87 % de 'no cae', decir siempre 'no' da 87 %	Matriz de confusión
Clustering	Perfiles semanales de 355 series	Silhouette, tamaño de grupos	Un número alto no garantiza grupos con sentido de negocio	Perfiles por grupo (lo revisa un humano)
Modelos de lenguaje	(fuera del alcance)	Jueces, evaluación humana, trazas	Probar con tres ejemplos elegidos por uno mismo	Trazas y conjuntos de evaluación

MLflow guarda las cuatro igual: parámetros, métricas, artefactos. Lo que cambia es qué métricas tienen sentido.

La trampa más grande: una métrica global esconde segmentos

Resultados reales sobre la tabla de oro, validación 1–18 de julio de 2026 (la reproducen en el notebook)

Una métrica global esconde segmentos: WAPE por tipo de mercado y por día



WAPE 4,8 %

global del baseline: parece un buen número

9,5 % lunes

el mismo baseline los lunes: dos lunes festivos (13 y 20 jul)

7,7 % lunes

con una regla de una línea: festivo = domingo (TODO 2)

Fuente: tabla de oro features_demanda_diaria; split temporal 30-jun; cálculo propio con src/xm_demanda/model

RECESO

10 minutos

Al volver: qué ofrece Databricks · anatomía del tracking · evaluar sin engañarse · registro en Unity Catalog

Mientras tanto: Git folder en main → Pull, y rama feature/c09-mlflow creada

iData
Escuela de Data & AI

¿Qué ofrece Databricks? Una pieza por paso, un solo catálogo

Lo marcado 'hoy' se ve en el notebook; todo funciona en Free Edition salvo AutoML

Qué ofrece Databricks para cada paso del ciclo (y qué de eso usamos hoy)

Todo vive en el mismo sitio que los datos y con los mismos permisos de Unity Catalog. Lo marcado "hoy" se ve en acción en el notebook.

hoy

MLflow Tracking

EXPERIMENTAR

Experimentos y runs: parámetros, métricas, artefactos, datasets, tags. Autolog para sklearn, LightGBM, XGBoost, PyTorch, transformers…

Interfaz para comparar runs lado a lado y graficar métricas.

Free Edition: sí, completo

hoy

Evaluación

EVALUAR

`mlflow.evaluate` con métricas estándar por tipo de tarea, más las tuyas (WAPE). `search_runs` para armar la tabla de comparación en código.

Para modelos de lenguaje: jueces y evaluación de agentes (fuera del alcance del curso).

Free Edition: sí

hoy

Model Registry en UC

REGISTRAR

Modelos con nombre de tres partes, versiones, alias, firma, permisos y linaje con tablas y runs. Un modelo es un objeto del catálogo.

Feature Engineering: lookups automáticos por llave al servir.

Free Edition: sí (sin lookups automáticos)

Model Serving

DESPLEGAR

Endpoint REST desde una versión o alias, con escalado a cero, logs de peticiones y AI Gateway para gobernar. Batch con `spark_udf` sobre millones de filas.

Apps para ponerle una cara al modelo.

Free Edition: Serving CPU y 3 apps · clase 11

Lakehouse Monitoring

MONITOREAR

Perfil de la tabla de inferencias: deriva de features, error real vs. pronóstico cuando llega el observado, alertas.

Clase 15

Jobs y Bundles

REPETIR

Reentrenar como una tarea programada con el mismo código; promover por ambiente con bundles. Lo que vuelve el ciclo un ciclo.

Clases 12–13

AutoML y Assistant

ACELERAR

AutoML: baseline y notebook generado para problemas tabulares, con sus runs en MLflow. Assistant: explicar errores, proponer código. Útiles con verificación (clase 8).

AutoML no está en Free Edition

Mosaic AI (vista rápida)

MÁS ALLÁ DEL CURSO

Vector Search, Agent Framework, Foundation Model APIs, fine-tuning: el mismo tracking y registro para modelos de lenguaje y agentes.

Mismo MLflow, mismo catálogo

Lo que importa del mapa: el tracking, el registro, los datos y los permisos son el mismo sistema. Un modelo registrado sabe de qué versión de qué tabla salió, y quien no puede ver la tabla tampoco puede usar el modelo.

La diferencia no es la lista: es que tracking, registro, datos y permisos son el mismo sistema. Quien no puede leer la tabla no puede usar el modelo.

Fuente: docs.databricks.com/aws/en/mlflow · /machine-learning/manage-model-lifecycle · MLflow nació en Databricks (2018) y es código abierto

Anatomía del tracking: experimento, runs y qué guarda cada uno

MLflow no entrena: anota. Cada intento deja todo lo necesario para volver a producir su resultado

Anatomía del tracking: experimento, runs y lo que cada run guarda

MLflow no "entrena": anota. Cada vez que pruebas algo, abres un run y dejas constancia de todo lo que haría falta para volver a producir ese resultado.

Experimento · demanda_7d

Una carpeta de intentos comparables sobre la misma pregunta. Cada fila es un run.

- **run · naive_estacional** — wape 0.048; tipo = baseline
- **run · naive_festivo** — wape 0.043; tipo = baseline
- **run · lightgbm_global** — wape 0.050; n_estimators 600
- **run · lightgbm_fuga** — wape 0.024 ⚠; tag fuga = true
- **run · clasificador_caida** — auc 0.81
- **run · kmeans_perfiles** — silhouette 0.31

Un run, por dentro

- **Parámetros** — lo que decidiste: lags, horizonte, n_estimators, learning_rate, corte temporal
- **Métricas** — lo que obtuviste: wape, mae_gwh, wape_regulado, wape_festivos… y por paso, si entrenas por épocas
- **Artefactos** — archivos: el modelo serializado, la gráfica de importancia, la matriz de confusión, el CSV de errores por serie
- **Dataset (log_input)** — qué datos: tabla, versión Delta, filas, esquema. Es el enlace con la clase 6: time travel
- **Tags y origen** — commit de Git, notebook, usuario, fuga=true, candidato=si. Lo pone MLflow solo en un Git folder
- **Modelo + firma** — el modelo con su esquema de entrada y salida y un ejemplo: lo que se registra en Unity Catalog

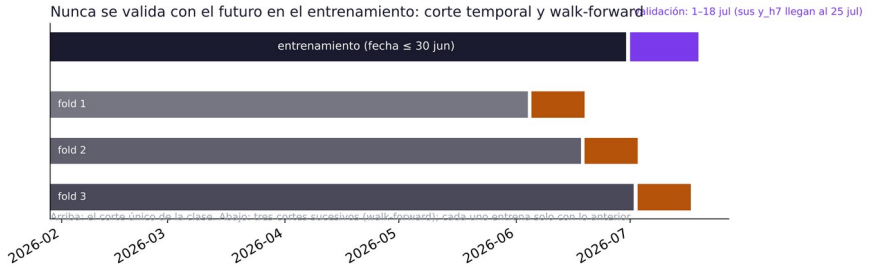
La prueba del run bien hecho: dentro de seis meses, sin hablar contigo, alguien abre el run y puede responder qué datos, qué código, qué parámetros y qué resultado. Si para contestar hay que buscar en un chat o en tu memoria, faltó algo en el run.

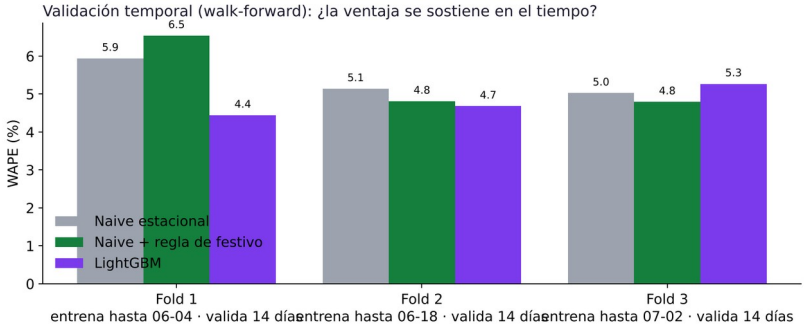
lo que hace correr_experimento() en el notebook

```
with mlflow.start_run(run_name="lightgbm_global"):
    mlflow.log_params({"modelo": "lightgbm_global", "horizonte_dias": 7, "corte": "2026-06-30", "n_estimators": 600})
    mlflow.log_metrics({"wape": 0.0498, "mae_gwh": 0.034, "wape_regulado": 0.040})
    mlflow.log_input(mlflow.data.load_delta(table_name=T_FEATURES, version="7"), context="training")
    reg.registrar_en_run(estimador, ejemplo, code_paths=[".../src/xm_demanda"])   # modelo + firma + código
```

Evaluar sin engañarse: corte temporal y walk-forward

Con series de tiempo nunca se baraja; y una sola validación es una muestra de tamaño uno





Lo que muestra la derecha

Fold 1: LightGBM gana claro (4,4 vs 5,9). Fold 3: pierde (5,3 vs 5,0). Con un solo corte, en una fecha o en la otra, las conclusiones serían opuestas. Promedio: LightGBM 4,8 · regla de festivo 5,4 · naive 5,4.

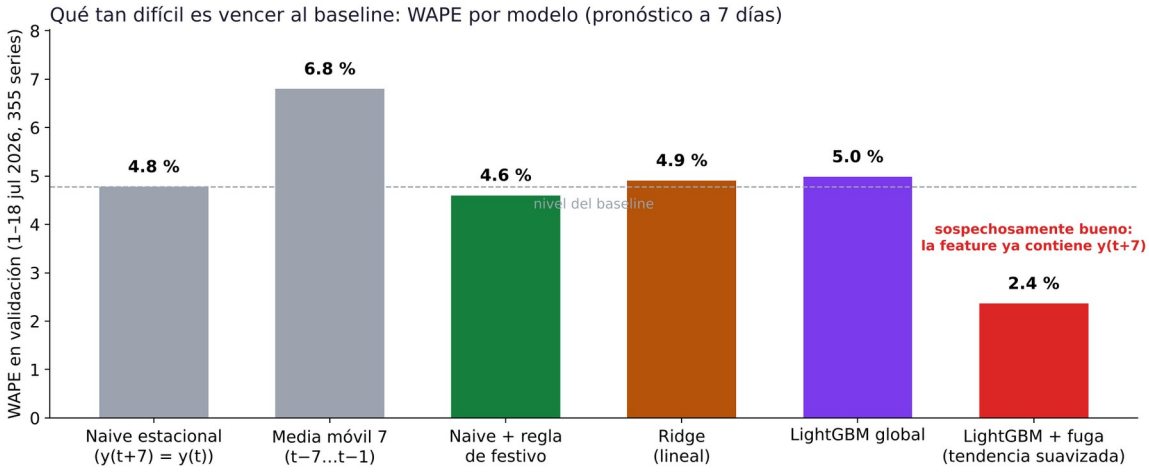
En MLflow: runs anidados

Un run padre walk_forward con los promedios; un hijo por fold con su corte y sus métricas. En la interfaz se ven agrupados.

Fuente: Hyndman & Athanasopoulos, Forecasting: Principles and Practice, 5.10 Time series cross-validation (otexts.com/fpp3/tscv.html)

El resultado incómodo: el baseline es muy difícil de vencer

WAPE en validación, 355 series, pronóstico a 7 días. Mismo corte, mismos datos, mismo tracking para todos



Sin baseline al lado…

un WAPE de 5 % parece un logro. Con baseline, es una pregunta: ¿qué aporta el modelo y dónde?

La barra roja

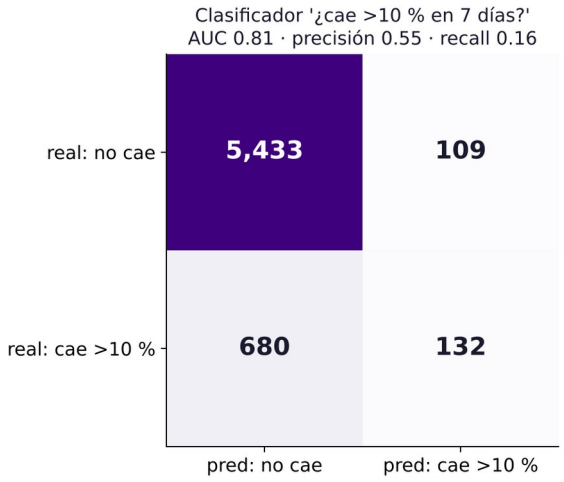
2,4 %: una 'tendencia suavizada' centrada contiene y(t+7). Un run demasiado bueno es una alarma.

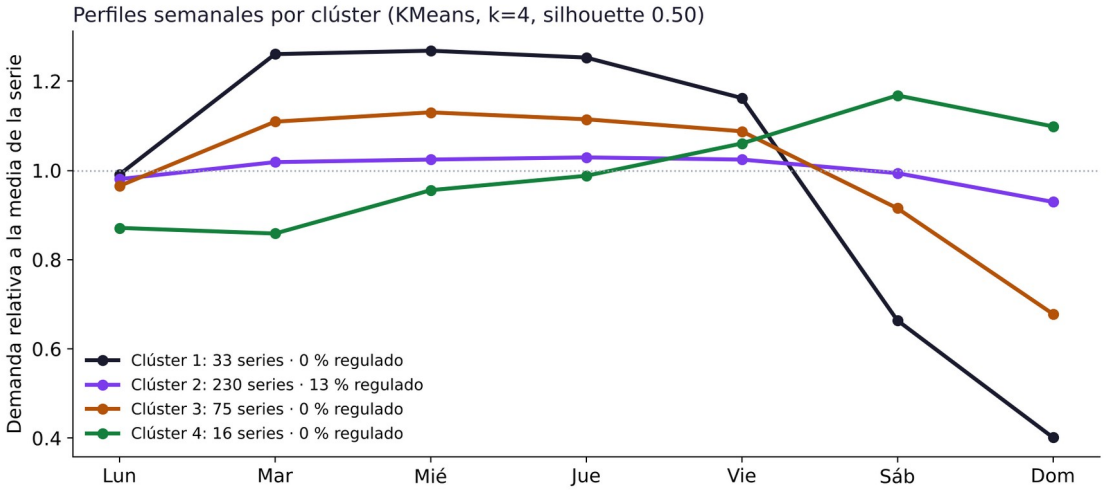
Por qué pasa en demanda eléctrica: la semana es muy marcada y seis meses de historia dan poco más de lo que ya da copiar la semana anterior. Por eso los operadores publican siempre contra un baseline estacional.

Fuente: cálculo propio sobre features_demanda_diaria; Makridakis et al., M5 accuracy competition (International Journal of Forecasting, 2022)

Otros tipos de modelo, el mismo tracking

MLflow no sabe de demanda: sabe de parámetros, métricas y artefactos. Mismos datos, preguntas distintas





AUC 0,81 · recall 0,16

Detecta 132 de 812 caídas con umbral 0,5. Bajar el umbral es una decisión de negocio.

Silhouette 0,50 · cuatro comportamientos sin decirle nada de sectores

Oscuro: industria de lunes a viernes (domingo −60 %). Morado: el grueso, plano, incluido todo el regulado. Verde: sube el fin de semana (comercio, entretenimiento).

Del run al modelo registrado: versiones, alias y linaje

Un run es un intento; un modelo registrado es un activo: nombre de tres partes, dueño, permisos, versiones

Del run al modelo registrado: versiones, alias y linaje en Unity Catalog

Un run es un intento; un modelo registrado es un activo de la empresa, con nombre de tres partes, dueño, permisos y versiones. El alias dice cuál está en uso sin que nadie tenga que cambiar código.

dev.models.demanda_7d

- **versión 3** `@champion` lightgbm_global · wape 0.050 · hoy
- **versión 2** `@challenger` naive_festivo · wape 0.043 · hoy
- **versión 1** `sin alias` naive_estacional · wape 0.048 · hoy

Quien consume escribe **models:/dev.models.demanda_7d@champion**. Promover = mover el alias. Volver atrás = mover el alias. Cero despliegues de código.

Lo que trae cada versión

Firma: esquema de entrada (las 24 columnas del contrato de oro) y de salida (kWh). Si alguien manda una columna de menos, falla antes de predecir, no después.

Ejemplo de entrada: una fila real, para probar el endpoint sin pensar.

Linaje: el run que lo produjo → su dataset (tabla + versión Delta) → su commit de Git → sus parámetros. Las cuatro versiones de la clase 8, enlazadas.

Permisos: quién puede leerlo, quién puede promoverlo. Es un objeto más de Unity Catalog, como una tabla.

1 · log_model con firma

Dentro del run: el modelo envuelto (pyfunc) con su preprocesamiento, para que reciba filas de oro y devuelva kWh.

2 · register_model

El run se convierte en versión N del modelo con nombre catálogo.esquema.modelo. Mismo artefacto, ahora gobernado.

3 · set_registered_model_alias

champion para el que sirve; challenger para el que compite. La clase 11 consume @champion sin saber qué versión es.

4 · "deploy code, not models"

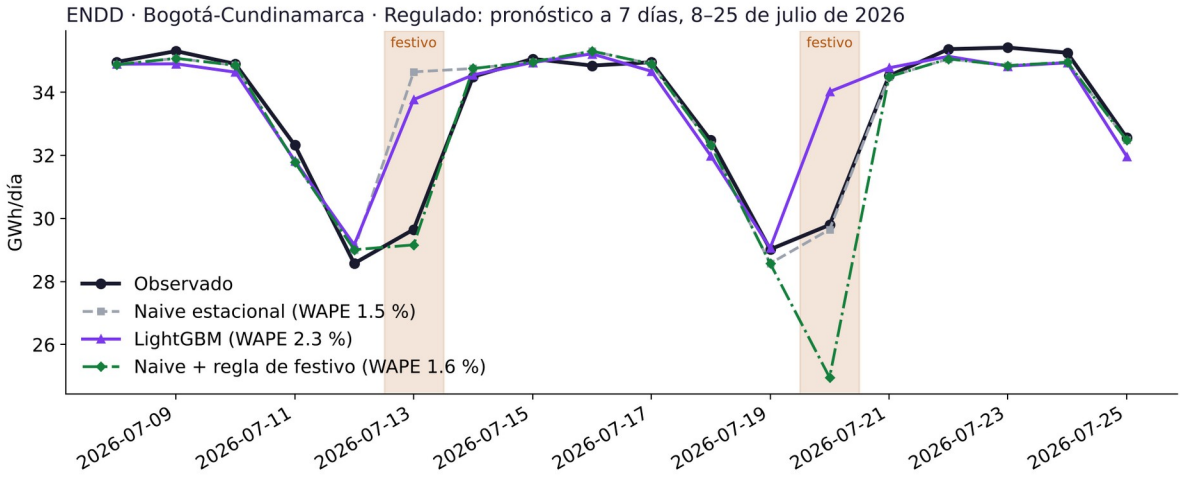
En XM el código de entrenamiento se promueve entre ambientes y cada ambiente entrena su versión con sus datos; el artefacto no viaja de dev a prod.

src/xm_demanda/model/registro.py

```
v = mlflow.register_model(f"runs:/{run_id}/modelo", "workspace.c01_docente.demanda_7d")
MlflowClient().set_registered_model_alias("workspace.c01_docente.demanda_7d", "champion", v.version)
# el consumidor (clase 11) nunca sabe qué versión es:
modelo = mlflow.pyfunc.load_model("models:/workspace.c01_docente.demanda_7d@champion")
# promover = mover el alias · volver atrás = mover el alias · cero cambios en el consumidor
```

Sector: el pronóstico es público, y el día difícil es el raro

XM, ENTSO-E y AEMO publican sus pronósticos de demanda; terceros los usan para decidir en el mercado



1,5 %

WAPE del baseline en la serie más grande (Bogotá regulado) en días normales y festivos

1,0 %

con la regla de festivo: acierta los dos lunes del 13 y el 20 de julio

Mensaje para XM: el día normal lo resuelve cualquier método; lo difícil son festivos, puentes y eventos. Un sistema que no mide por segmento nunca lo va a ver, y uno que no registra no lo puede explicar.

Fuente: xm.com.co (pronóstico oficial de demanda) · transparency.entsoe.eu (day-ahead total load forecast) · aemo.com.au (forecast accuracy reports) · cálculo propio

Free Edition vs. Azure Databricks: qué simulamos hoy

MLflow y Unity Catalog son los mismos; cambian dónde viven los objetos y quién promueve

Elemento	Hoy en Free Edition	En XM sobre Azure
Experimento	En tu carpeta de usuario	Uno compartido por ambiente: /Shared/xm-demanda/{dev,qa,prod}
Modelo registrado	workspace.c01_<usuario>.demanda_7d	{dev,qa,prod}.models.demanda_7d, cada uno entrenado por su job
Entrenamiento	Celdas del notebook	notebooks/04_train.py como tarea del job train_model (clase 12)
Promoción a @champion	La hace el notebook según el walk-forward	Aprobación humana o regla explícita; registrada como evento
Inferencia	Una celda con load_model(@champion)	Job batch y endpoint de Model Serving (clase 11)

Notebook y lab: siete runs, dos TODO, un modelo registrado

clases/c09-mlflow/notebooks/lab09_mlflow_registro · widget usuario · requiere features_demanda_diaria (lab 8)

1

Secciones 1–3: corte temporal y primer run

TODO 1: wape en metricas.py · naive_estacional a mano

2

Secciones 4–5: modelo global y comparación

lightgbm_global · search_runs · error por día y mercado

3

Sección 6: TODO 2, la regla de festivo

NaiveFestivo.fit en modelos.py: un factor por serie

4

Secciones 7–9: walk-forward, fuga, otros tipos

Runs anidados · tag fuga=true · clasificador y KMeans

5

Sección 10 + verificar()

Dos versiones con @champion y @challenger · inferencia

Criterios de aceptación (hito 5 abierto)

- test_model.py sin skip
- 7 runs + 3 hijos de walk-forward
- Runs de pronóstico con dataset versionado y firma
- demanda_7d con @champion y @challenger
- Dos preguntas; verificar() en OK; PR en verde

Comparar con s09

github.com/<docente>/Databricks-to-XM/compare/s09...<tú>:Databricks-to-XM:main · o git diff s09 -- src/xm_demanda/model clases/c09-mlflow/notebooks

Duración: 45 min guiado + 30 min solo

Así sería en Azure: el mismo MLflow, con aprobaciones y ambientes

Hoy	En XM
Siete runs en un experimento personal	Experimentos por ambiente; cada job de entrenamiento crea un run con commit, configuración y versión Delta
Baseline calculado en el notebook	El baseline es un run más en cada entrenamiento: el modelo nuevo solo se promueve si lo vence en walk-forward
Alias movidos por el notebook	Promoción con aprobación (humana o regla) y rastro; rollback = mover el alias
Linaje consultado a mano	Catalog Explorer → modelo → Lineage: run, tabla, versión, notebook. Respuesta a un auditor en minutos
deploy code, not models	El código de 04_train.py viaja de dev a prod por el bundle; el artefacto no

Lo que no cambia: runs comparables, baseline al lado, métricas por segmento y un alias que separa al que sirve del que compite.

Resumen de las clases 9 y 10 y preparación para la clase 11

Logros de hoy

MLOps es el ciclo

No el entrenamiento: datos → … → reentrenar, sin miedo

Medir es para decidir

Cada métrica con su para qué, siempre por segmento

Tracking hace todo comparable

Regresión, clasificación, clustering: mismo run

Baseline primero

Y un run demasiado bueno es una alarma

Registro con alias

@champion desacopla al que entrena del que consume

Próxima clase

Inferencia batch

Job que escribe gold_energia.pronostico_demanda con @champion

Model Serving

Endpoint en tiempo real desde el modelo registrado

Databricks Apps

Observado vs. estimado por segmento

Traer a la clase 11

Lab 9 terminado: la clase 11 consume su modelo



iData
Global | Data & AI

Gracias

Nos vemos en la clase 11: Inferencia batch, Serving y Apps

DOCENTE

Manuela Larrea Gómez